Assignment 1 - Part A: Sorting Algorithms

Name: Tina Nguyen
Course: COMP-SCI 5592 - Design & Analysis of Algorithms
Semester: Fall 2026

1. What I did

For Part A, I compared TimSort, Quicksort, and Introsort using the provided file, *Most English 50000 Words.txt*. I wanted to see how the algorithms changed when I gave them more words to sort. I used the first 25%, 50%, 75%, and 100% of the file, so each algorithm worked on the same words for each test.

Treatment	Number of words
25%	12,500
50%	25,000
75%	37,500
100%	50,000

2. How I tested the algorithms

I ran the tests in Python on Google Colab. TimSort used Python's built-in sort with counting for comparisons. For Quicksort, I used a median-of-three pivot and three-way partitioning. Introsort started like Quicksort and could switch to heapsort if the recursion got too deep. I measured comparisons, pairwise swaps, and the real elapsed time. I ran the time test five times and used the median time. I also checked that every output was sorted correctly.

One thing to keep in mind: TimSort moves data in a different way, so I wrote **N/A** for its pairwise swaps instead of calling them zero. Also, these times are from one test environment, so a different Colab session may show different numbers.

3. Results table

Size	Algorithm	Comparisons	Swaps	Time (ms)
25%	TimSort	154,085	N/A	37.90
25%	Quicksort	272,263	136,489	46.63
25%	Introsort	272,263	136,489	47.39
50%	TimSort	333,008	N/A	77.08
50%	Quicksort	592,522	291,860	92.18
50%	Introsort	592,522	291,860	92.68
75%	TimSort	522,033	N/A	120.31

Size	Algorithm	Comparisons	Swaps	Time (ms)
75%	Quicksort	883,750	426,149	141.75
75%	Introsort	883,750	426,149	142.58
100%	TimSort	715,725	N/A	172.91
100%	Quicksort	1,240,052	597,804	201.81
100%	Introsort	1,240,052	597,804	202.88

From my results, TimSort had the lowest median running time at every data size. The two versions of Quicksort and Introsort were pretty close on the original data. For example, at 50,000 words, TimSort took about 172.91 ms, Quicksort about 201.81 ms, and Introsort about 202.88 ms.

4. Line charts

I made these graphs to compare how the results changed when the number of words increased.

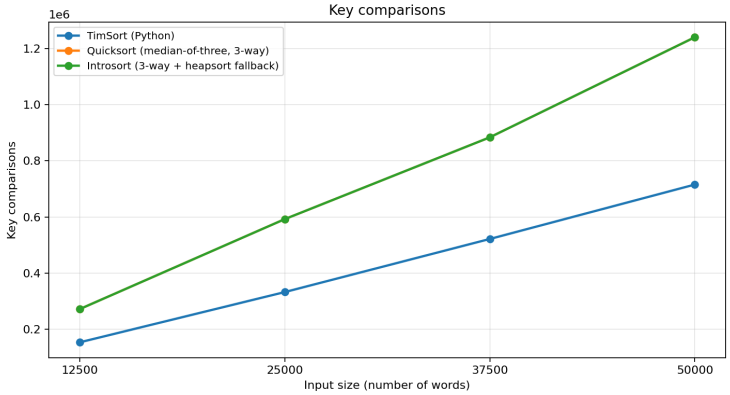


Figure 1. Number of comparisons

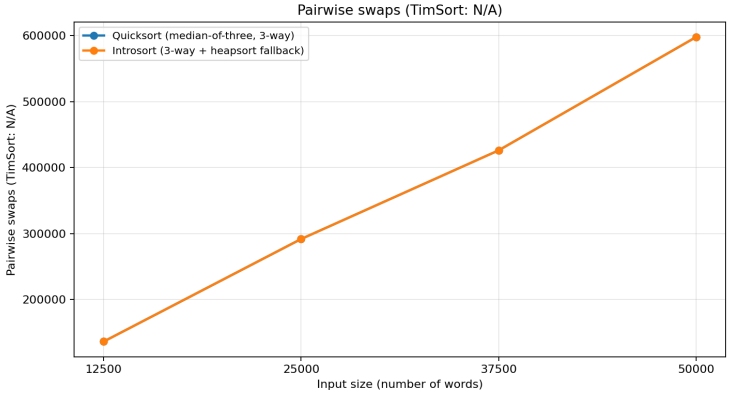


Figure 2. Number of pairwise swaps

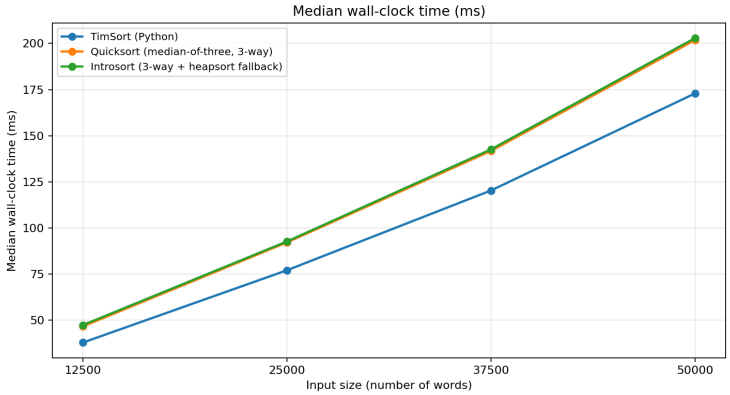


Figure 3. Median execution time

The graphs show that comparisons and running time went up as I used more words. The swap chart only compares Quicksort and Introsort, because my TimSort measurement does not count pairwise swaps.

5. Space complexity

Space complexity is how much extra memory the algorithm may need while sorting. Here is what I found for the sorting methods used in my notebook:

Algorithm	Extra space
TimSort	Up to O(n) temporary storage
Quicksort	O(log n) recursive stack in this implementation
Introsort	O(log n) recursive stack in this implementation

My experiment also made a copy of the input list before sorting, so that adds O(n) space for each test. The table is about the sorting method itself, not the whole experiment.

6. Scalability

To check scalability, I compared each algorithm at 12,500 words and 50,000 words. The number of words became four times bigger, and I wanted to see how much the execution time increased.

Algorithm	12,500 words	50,000 words	Time increase
TimSort	37.90 ms	172.91 ms	4.56x
Quicksort	46.63 ms	201.81 ms	4.33x
Introsort	47.39 ms	202.88 ms	4.28x

All three algorithms took longer as the data size increased, which makes sense because they had more words to compare. In this test, they all scaled reasonably, but TimSort stayed the fastest. I would not say it always wins, because the input order and the exact implementation can change the results.

7. Adaptive nature (different word orders)

For this extra test, I used 12,500 words and tried four input orders: original, already sorted, reverse sorted, and shuffled. I wanted to see if the algorithms worked better when the words were already in order.

Input order	TimSort (ms)	Quicksort (ms)	Introsort (ms)
Original	38.82	47.30	47.48
Sorted	5.82	205.13	137.34
Reverse	5.64	148.94	121.44
Shuffled	36.52	45.38	47.63

TimSort got much faster when the words were already sorted or in reverse order. I think this shows its adaptive nature, because it can take advantage of runs that are already ordered. Quicksort was much slower on those same inputs in my test. Introsort handled those cases better than Quicksort here because it can fall back to heapsort.

8. My conclusion

After doing this assignment, I learned that sorting speed depends on both the number of items and the order they start in. TimSort performed best on the dataset I used, especially for already sorted and reverse sorted words. Quicksort and Introsort had similar results on the original file, but Introsort did better for the tricky input orders in my extra test. I also learned why checking comparisons, swaps, time, and space gives a better picture than looking at only one number.

Files used: Most English 50000 Words.txt; sorting_results.csv; adaptive_results.csv; comparisons.png; swaps.png; runtime.png.